IRE PROJECT REPORT

News Article Personalized Ranking System & A/B Testing

IIITH – Monsoon

Darmoju Deekshitha - 2024900005
Sohan Mupparapu - 2022101061

1. Summary

This report presents a learning-to-rank (LTR) system for personalized news article recommendation. The goal is to optimize user engagement by ranking news articles according to individual user preferences rather than relying solely on generic relevance scoring. The system combines multiple ranking approaches and incorporates A/B testing to evaluate improvements against a baseline Elasticsearch ranking.

Through experimentation with 700 queries (500 baseline + 200 A/B test queries), the system demonstrates that personalization strategies informed by user interactions such as clicks, dwell time, likes, shares, and bookmarks can significantly improve engagement metrics. Position bias mitigation and efficient feature extraction further enhance ranking effectiveness.

Code base: https://github.com/darmojud/IRE/tree/Project/src

2. Introduction

News platforms often struggle to deliver content that matches individual user interests. Traditional ranking methods, such as keyword-based or relevance-only search, fail to account for diverse user preferences. As a result, users may encounter irrelevant articles, reducing engagement metrics such as clicks, dwell time, and content sharing.

This project addresses three main challenges:

1. **Personalized Ranking** – Develop algorithms that dynamically adjust rankings based on a user's historical and current behaviour.
2. **Position Bias Mitigation** – Correct for the tendency of users to interact more with top-ranked items, regardless of relevance.
3. **Statistical Validation** – Rigorously evaluate ranking improvements using A/B testing methodology.

The system is built on simulated user interaction data and news article content:

- **Article Dataset (**`data/articles.jsonl`**)** : JSONL format with fields including `uuid`, `text`, and `topics`.
- **User Simulation Platform**: REST API providing two endpoints:
  - `/query`: Returns queries for a user (`user_id` and `query_text`).
  - `/ranklist`: Accepts ranked article lists and returns simulated user actions.
- **User Actions Captured**:

    - **Click**, **Like**, **Share**, **Bookmark**, **Dwell Time** (duration)
- **Hidden Features**: Users have latent preferences that are not directly observable through article metadata, requiring models to infer interests from behavioural data.

3. System Architecture

3.1. Core Components

- **Feature Extraction**
    - Converts article content, query context, and user history into numeric features suitable for ranking models.
    - Features include textual similarity and user-specific engagement signals.
- **Ranking Models**
    - **Baseline:** Uses Elasticsearch relevance scores based on query-article matching without personalization.
    - **Collaborative Filtering:** Ranks articles by leveraging user-user similarity and past interaction patterns to predict relevant content.
    - **Learning-to-Rank (LTR):** LightGBM LambdaMART models trained on user feedback (clicks, dwell time) to optimize ranking metrics like nDCG.
    - **XGBoost Ranking:** Gradient-boosted decision trees that incorporate article features, user embeddings, and interaction history to produce a personalized ranking score.
- **Position Bias Correction:** Implements inverse propensity weighting and normalization techniques to account for the higher visibility of top-ranked articles.
- **A/B Testing Framework**
    - Splits queries into baseline and experimental groups to statistically validate the effect of personalization.
    - Monitors engagement metrics: click-through rate, dwell time, likes, shares, bookmarks.
- **API Integration**
    - Queries are fetched from the `/query` endpoint.
    - Ranked results are submitted to `/ranklist` for real-time feedback collection.

3.2. Ranking Workflow

1. Receive a query and user ID from the simulation API.
2. Retrieve candidate articles from Elasticsearch.
3. **Stage 1 – Initial Ranking:**
    - Depending on the selected variant, rank candidates using:
        - **Baseline:** Elasticsearch relevance scores
        - **Collaborative:** SVD-based user-item embeddings
        - **LTR:** LightGBM LambdaMART model
        - **XGBoost:** Trained XGBoost ranking model
    - This produces an initial ranking independent of the article positions.
4. **Stage 2 – Position-Aware Feature Extraction:**
    - Re-extract features for each article **including its actual position** in the ranked list.
    - These features are used for further training (e.g., IPS) and evaluation.
5. Submit the final ranked list to the `/ranklist` endpoint.

6. Collect user feedback (clicks, likes, dwell, shares, bookmarks) for iterative model training and performance evaluation.

3.3. Evaluation & Feedback (A/B Testing):

- **Phase 1:** Collect baseline metrics using initial queries with the default ranking.
- **Phase 2:** Train models (Collaborative, LTR, XGBoost) using historical user interaction data.
- **Phase 3:** Run A/B test with multiple variants across new queries; collect clicks, dwell, likes, shares, bookmarks.
- **Phase 4:** Analyse results with statistical tests to measure improvement over baseline, enabling iterative optimization of ranking strategies.

4. Experimental Results

The A/B testing framework evaluated four ranking variants—**Baseline**, **Collaborative Filtering**, **Learning-to-Rank (LTR)**, and **XGBoost**—over multiple phases, collecting **14,000 interaction logs** from **197 unique users**.

4.1. Baseline Performance (Phase 1)

500 initial queries were routed exclusively to the baseline model to collect training data. Key metrics stabilized around:

- **CTR:** ~0.43%–0.55%
- **Avg Dwell Time:** ~0.7–1.0s
- **Weighted Clicks:** ~0.21
- **Impressions:** 10,000 (over 500 queries)

These logs were used to train the LTR, Collaborative Filtering, and XGBoost models.

4.2. Model Training (Phase 2)

- **Collaborative Filtering:** trained on *190 users × 4599 articles*
- **LTR Model:** trained with *10,000 samples*, 100 iterations of LambdaMART
  - Final loss stabilized at **0.0648**
- **XGBoost Model:** trained on same 10,000 samples

All models were successfully saved for deployment.

4.3. A/B Testing Performance (Phase 3)

Across 200 experiment queries, variants were randomly assigned:

Variant	Query Share
Baseline	30.5%
Collaborative	28.5%

Variant	Query Share
LTR	24.0%
XGBoost	17.0%

Overall Metrics (Phase 3 Summary)

Model	CTR	Avg Dwell Time	Weighted Clicks
Baseline	0.57%	0.17 s	0.1870
Collaborative	0.26%	**0.42 s**	**0.2400**
LTR	**0.62%**	0.13 s	0.1726
XGBoost	0.15%	0.01 s	0.0413

4.4. Statistical Significance Testing (Phase 4)

Each variant was compared against Baseline using two-sample hypothesis tests.

LTR vs Baseline

- CTR improvement: **+8.93%**
- Weighted clicks: **–7.66%**
- **p-values > 0.8** → *Not statistically significant*

XGBoost vs Baseline

- CTR dropped by **74%**
- Weighted clicks dropped **78%**
- Model underperformed Baseline
- p-values ≈ 0.12–0.23 → *Not significant*

Collaborative Filtering vs Baseline

- Weighted clicks **+28.38%** (highest among variants)
- Dwell time increased **147%**
- CTR dropped from 0.57% → 0.26%
- p-value ≈ 0.78 → *Not significant*

4.5. Recommendation

Because no model demonstrated statistically significant improvement, the system should:

- **Continue using the baseline model**, or
- **Collect more interaction logs** before re-evaluating
- Consider **feature engineering improvements** for LTR and XGBoost

5. Conclusion

The A/B testing experiment evaluated four ranking strategies Baseline, Collaborative Filtering, LTR (Learning-to-Rank), and XGBoost using 14,000 simulated interaction logs across 197 unique users. The goal was to determine whether any advanced ranking model significantly improved user engagement metrics over the baseline Elasticsearch scoring.

5.1. Key Findings

- No model achieved statistically significant improvements over the baseline across CTR, dwell time, actions, or weighted clicks (p-value > 0.05 for all comparisons).
- Collaborative Filtering showed the highest weighted clicks (0.2400) and was the best overall variant, but the improvement was not statistically significant due to high variance and limited sample size.
- LTR matched the baseline performance on CTR but did not provide consistent improvements. Some gains were observed in early batches, but performance was unstable.
- XGBoost underperformed across all metrics, giving significantly lower CTR and weighted clicks.
- Baseline remained stable with consistent CTR and dwell time throughout the experiment.

5.2. Reasons for Non-Significant Results

1. **Sparse user interactions**: Very low actual engagement (almost zero likes/shares/bookmarks) limits the signal for ranking models to learn from.
2. **Short dwell times & low CTRs**: Models cannot rely on meaningful behavioral patterns when the underlying feedback is extremely noisy.
3. **Simulated data limitations**: Synthetic behavior does not capture real user preference patterns, making ML-model generalization weak.
4. **Insufficient data for LTR/XGBoost**: 10k training samples are not enough for high-dimensional feature-based ranking models.

While advanced models showed some directional improvements particularly collaborative filtering none of the variants produced statistically significant gains over the baseline.